



AutoMapper in ASP.NET Core

Object-to-Object Mapping — Step by Step Guide

🔗 AutoMapper কী?

AutoMapper একটি library যা একটি object এর data অন্য object এ automatically copy করে দেয়। সাধারণত Entity (Database model) থেকে DTO (Data Transfer Object) এ convert করতে ব্যবহার হয় — যাতে সব field manually assign না করতে হয়।

✗ AutoMapper ছাড়া	☑ AutoMapper দিয়ে
<pre>var dto = new EmployeeDto(); dto.Name = emp.Name; dto.Email = emp.Email; dto.Dept = emp.Department; // ... প্রতিটা field manually</pre>	<pre>var dto = _mapper.Map<EmployeeDto>(emp); // একটাই line – সব হয়ে গেল! 🚀</pre>

📦 Required NuGet Package

📦 Package	Purpose
AutoMapper	Core mapping library
AutoMapper.Extensions.Microsoft.DependencyInjection	ASP.NET Core DI integration (AddAutoMapper)



Install via PMC: Install-Package

AutoMapper.Extensions.Microsoft.DependencyInjection

🔗 Step 1 — Entity ও DTO Class তৈরি করো

Entity = Database এর actual model। DTO = Client কে যা পাঠাবে (sensitive field বাদ দিয়ে)।

```
// — Entity (Database Model) —————
public class Employee
{
    public int    Id        { get; set; }
    public string Name      { get; set; } = string.Empty;
```

```

    public string Email      { get; set; } = string.Empty;
    public string Department { get; set; } = string.Empty;
    public decimal Salary    { get; set; }    // sensitive – DTO তে রাখব না
}

// — DTO (Client কে পাঠানোর জন্য) —————
public class EmployeeDto
{
    public int    Id          { get; set; }
    public string Name        { get; set; } = string.Empty;
    public string Email       { get; set; } = string.Empty;
    public string Department  { get; set; } = string.Empty;
    // Salary নেই – intentionally বাদ দেওয়া হয়েছে
}

```

🔗 Step 2 — MappingProfile তৈরি করো

Profile class এ বলে দিতে হয় — কোন class থেকে কোন class এ map হবে:

```

using AutoMapper;

public class MappingProfile : Profile
{
    public MappingProfile()
    {
        // Employee → EmployeeDto (same name হলে auto match)
        CreateMap<Employee, EmployeeDto>();

        // Reverse map – DTO থেকে Entity তেও যেতে পারবে
        CreateMap<EmployeeDto, Employee>();

        // Shortcut – উপরের দুটো একসাথে:
        // CreateMap<Employee, EmployeeDto>().ReverseMap();
    }
}

```

💡 **Auto Match Rule:** Property নাম দুই class এ same হলে AutoMapper নিজেই match করে নেয়। আলাদা নাম হলে ForMember() দিয়ে বলে দিতে হয়।

🔗 Step 3 — Program.cs এ Register করো

AddAutoMapper() call করলে সব Profile automatically scan হয়ে register হয়:

```
var builder = WebApplication.CreateBuilder(args);

// AutoMapper register – Assembly scan করে সব Profile খুঁজে নেবে
builder.Services.AddAutoMapper(typeof(Program).Assembly);

// অথবা specific profile দিয়ে:
// builder.Services.AddAutoMapper(typeof(MappingProfile));

builder.Services.AddControllers();
// ... বাকি registration
```

🔗 Step 4 — Controller এ IMapper Inject ও Use করো

```
[ApiController]
[Route("api/[controller]")]
public class EmployeeController : ControllerBase
{
    private readonly AppDbContext _context;
    private readonly IMapper _mapper;

    // Constructor Injection
    public EmployeeController(AppDbContext context, IMapper mapper)
    {
        _context = context;
        _mapper = mapper;
    }

    // — GET All —————
    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var employees = await _context.Employees.ToListAsync();

        // List<Employee> → List<EmployeeDto>
        var dtos = _mapper.Map<List<EmployeeDto>>(employees);
        return Ok(dtos);
    }
}
```

```

    }

    // — GET Single —————
    [HttpGet("{id}")]
    public async Task<IActionResult> GetById(int id)
    {
        var emp = await _context.Employees.FindAsync(id);
        if (emp == null) return NotFound();

        // Employee → EmployeeDto
        var dto = _mapper.Map<EmployeeDto>(emp);
        return Ok(dto);
    }

    // — POST —————
    [HttpPost]
    public async Task<IActionResult> Create(EmployeeDto dto)
    {
        // DTO → Entity
        var employee = _mapper.Map<Employee>(dto);
        _context.Employees.Add(employee);
        await _context.SaveChangesAsync();
        return Ok(_mapper.Map<EmployeeDto>(employee));
    }
}

```

Step 5 — Custom Mapping (আলাদা নামের Field)

Source ও Destination এর property নাম আলাদা হলে ForMember() দিয়ে manually বলে দিতে হয়:

```

public class Employee
{
    public string FirstName { get; set; } = string.Empty;
    public string LastName { get; set; } = string.Empty;
    public string Dept { get; set; } = string.Empty;
}

public class EmployeeDto
{
    public string FullName { get; set; } = string.Empty; // আলাদা নাম
    public string Department { get; set; } = string.Empty; // আলাদা নাম
}

```

```
// MappingProfile এ:
CreateMap<Employee, EmployeeDto>()
    .ForMember(
        dest => dest.FullName,
        opt => opt.MapFrom(src => src.FirstName + " " + src.LastName))
    .ForMember(
        dest => dest.Department,
        opt => opt.MapFrom(src => src.Dept));
```

Step 6 — Nested Object Mapping

Nested object (related entity) থাকলে সেটার জন্যও আলাদা CreateMap() লিখতে হবে:

```
public class Employee
{
    public int      Id      { get; set; }
    public string   Name    { get; set; } = string.Empty;
    public Department Department { get; set; } = new(); // nested
}

public class Department
{
    public int      Id      { get; set; }
    public string   Name    { get; set; } = string.Empty;
}

public class EmployeeDto
{
    public string      Name      { get; set; } = string.Empty;
    public DepartmentDto Department { get; set; } = new(); // nested DTO
}

public class DepartmentDto
{
    public string Name { get; set; } = string.Empty;
}

// MappingProfile এ দুটো map লাগবে:
CreateMap<Department, DepartmentDto>();
CreateMap<Employee, EmployeeDto>(); // nested auto handle হবে
```

⚠ Important: Nested object এর map আগে define করতে হবে। AutoMapper নিজেই nested object গুলো recursively map করে নেবে।

⚡ Quick Reference — Important Methods

Method / Code	কাজ
<code>_mapper.Map<DestType>(source)</code>	Single object map করো
<code>_mapper.Map<List<DestType>>(sourceList)</code>	List map করো
<code>_mapper.Map(source, destination)</code>	Existing object এ map করো (update)
<code>CreateMap<Src, Dest>()</code>	Profile এ mapping define করো
<code>CreateMap<Src, Dest>().ReverseMap()</code>	দুই দিকেই map হবে
<code>ForMember(dest, opt => opt.MapFrom(src))</code>	Custom field mapping
<code>ForMember(dest, opt => opt.Ignore())</code>	নির্দিষ্ট field ignore করো
<code>AddAutoMapper(typeof(Program).Assembly)</code>	Program.cs এ সব Profile register

📋 Common Use Cases

Ignore একটি Field:

```
CreateMap<Employee, EmployeeDto>()
    .ForMember(dest => dest.Salary, opt => opt.Ignore());
```

Constant / Fixed Value দেওয়া:

```
CreateMap<Employee, EmployeeDto>()
    .ForMember(dest => dest.Status, opt => opt.MapFrom(src => "Active"));
```

Existing Object Update করা (PUT request এ কাজে লাগে):

```
// DB থেকে existing entity আনো
var existing = await _context.Employees.FindAsync(id);
```

```
// DTO এর data existing entity তে map করো
_mapper.Map(updateDto, existing);

await _context.SaveChangesAsync();
```

Recommended Folder Structure

```
YourProject/
├─ Controllers/
│   └─ EmployeeController.cs
├─ Models/
│   ├── Employee.cs          ← Entity
│   └─ Department.cs        ← Entity
├─ DTOs/
│   ├── EmployeeDto.cs       ← DTO
│   └─ DepartmentDto.cs     ← DTO
├─ Mappings/
│   └─ MappingProfile.cs     ← AutoMapper Profile
├─ Data/
│   └─ AppDbContext.cs
└─ Program.cs
```

☑ **মনে রেখো:** MappingProfile টা সবসময় Mappings/ folder এ রাখো এবং Program.cs এ `AddAutoMapper(typeof(Program).Assembly)` দিলে সব Profile auto-scan হয়ে যাবে — আলাদা করে register লাগবে না।